

발전소 Digital Twin

4족 보행 로봇 자율 점검

macOS + Docker arm64 · ROS 2 Jazzy · Gazebo Harmonic · Unitree Go2

프로젝트 한 문장 정의

우리는 macOS + Docker(arm64 네이티브) 위에서 ROS 2 Jazzy와 Gazebo Harmonic으로 발전소 Digital Twin을 구축하고, 4족 보행 로봇이 험지를 스스로 통과하고(Locomotion), 아날로그 계기와 과열 배관을 눈으로 읽어내며(Inspection AI), 가스 누출원을 추적한 뒤 안전하게 복귀하는(Gas/Safety) 자율 점검 시스템을 개발하여, 그 모든 상태와 결과를 웹 관제 화면에 실시간 시각화한다.

필수 205h

보너스 60h

Phase 0 실측 후 스택 변경

계획 → 실측 → 근거 있는 변경

도훈

Locomotion

운학

Inspection AI

태우

순찰 경로 · Critical Path

수현

Virtual Plant · 관제

채현

Gas / Safety

핵심 문제

사람이 들어가기 위험한 구역일수록 점검 주기가 짧고, 계기는 디지털 통신이 안 되는 아날로그 압력계다.
→ 작업자 안전 리스크 · 데이터의 비정형·비실시간성 · 이상 징후 조기 발견 실패
해결: 사람 대신 4족 보행 로봇을 보내고, 결과를 Digital Twin 관제 화면에서 실시간으로 확인한다. (가상 발전소에서 전 과정 검증)

Stakeholder

구분	대상	기대하는 것
1차	발전소 정비팀	위험 구역 무인 점검, 계기 수치 자동 기록
2차	로봇솔루션팀	실기 투입 전 보행·인식 알고리즘 검증 환경
3차	DX추진팀	설비 상태 데이터의 디지털화 및 관제 통합

최종 결과물

module1_locomotion	험지 보행 · MPC · Fall Recovery
module2_inspection	게이지 독해 · 열화상 퓨전
module3_gas_safety	플룸 · Source Seeking · RTH
simulation / dashboard	월드 SDF · 웹 관제
docs	REPORT.md · README.md · LI

모델링 최소 조건 · GitHub Repository 1식

복도 45 × 3.5 × 4 m	공장 25 × 18 × 7 m	험지 방해물 복도 3 · 공장 5	로봇개 Unitree Go2 × 1	공장기계 1기 + 가스탱크
----------------------	---------------------	-----------------------	------------------------	-------------------

실제 로봇 하드웨어는 쓰지 않는다. 가상 발전소(Virtual Plant)에서 보행·인식·가스 추적·관제까지 전 과정을 구현하고 정량 지표로 검증한다.

Digital Twin 4요소 ↔ 우리 구현물

DT 구성 요소	요구사항이 정의한 역할	우리 구현 (Harmonic)	실제 발전소 반영
Virtual Plant	모든 데이터가 적용되는 가상 환경	simulation/worlds/ 복도·공장 SDF, 계단, 파이프 잔해, 공장기계, 가스탱크 + heat signature	공간 치수, 통행 폭, 계단 경사, 장애물 밀도, 조명, 배관 표면 온도
Robot Twin	로봇 상태 및 센서 데이터 생성	Go2 URDF + gz_ros2_control gpu_lidar / rgbd / thermal / IMU 가스 센서(자체 노드)	관절 구성, 센서 장착 위치·시야각, 배터리 소모
Inspection Data	로봇이 수집하는 설비 상태 데이터	/inspection/gauge_reading /inspection/thermal_alert /gas/concentration	아날로그 계기 수치, 배관 표면 온도(K), 가스 농도 분포
Monitoring System	분석 결과 및 로봇 상태 시각화	dashboard/ rosbridge + 웹 대시보드 (보너스①을 필수에 흡수)	로봇 위치, 임무 상태, 점검 이력, 가스 농도맵

발표에서 한 줄로

Virtual Plant는 월드와 온도 태그, Robot Twin은 Go2와 센서(+thermal), Inspection Data는 게이지·열화상·가스 토픽, Monitoring System은 rosbridge 웹 대시보드다. 네 요소가 각각 폴더와 토픽에 대응하므로 “역할만 나열한 팀”과 같린다.

Phase 0 실측 → 스택을 바꿨다

개발 머신이 macOS(Apple Silicon)라 ROS 2를 네이티브로 돌릴 수 없다.
우회가 아니라 부가 목표로 전환: 누구나 docker compose up 한 번으로 재현 가능한 무료 환경. 확정 = 계획이 아니라 Phase 0 실측 이후.

	당초 계획	실측 결과 (2026-08-31)	최종 확정
ROS 배포판	Humble	humble-desktop은 amd64 전용 에뮬레이션 빈 월드 RTF 0.5	Jazzy
시뮬레이터	Gazebo Classic	arm64에 Classic도 ros-gz-sim도 없음	Gazebo Harmonic (gz-sim8)
근거	—	Jazzy+Harmonic은 arm64에서 ros-gz-sim, gz_ros2_control, Nav2, Foxglove 제공	arm64 네이티브 실행

불리 — 시간 배분을 바꿈

CHAMP 등 Humble 기준 4족 오픈소스는 Jazzy 포팅 검증이 필요하다.
→ 리스크 1순위. stage1을 1-A / 1-B로 이중화했다. (슬라이드 6)

유리 — 작업을 줄임

Harmonic에 thermal 카메라가 네이티브다 (type="thermal").
Classic 기준의 “열화상 합성 플러그인 자작”을 취소했다. 8h → 2h.

이 의사결정 기록 자체가 ①의 가장 강한 발표 재료다. “계획대로 실행했다”가 아니라 계획 → 실측 → 근거 있는 변경.
전체-1(가장 어려웠던 기술적 문제)의 1순위 후보: Apple Silicon 실행 환경 + Jazzy/Harmonic 전환. 실측 데이터(RTF 0.5)를 보유한다.

의존 관계 — P2는 모듈이 아니라 인프라

P0 개발환경 Docker arm64 / ROS 2 Jazzy / Gazebo Harmonic / Foxglove ← 없으면 아무것도 실행 불가

P1 Virtual Plant + Robot Twin 월드 SDF · 온도 태깅 · Go2 URDF · 센서 + ros_gz_bridge

P2 Module 1 · Locomotion Terrain Mapping → stage1 보행 → stage2/3 MPC → Fall Recovery ★ 55h 병목

여기서부터 병렬

P3 Module 2 Inspection AI

게이지 · 열화상-RGB 정합
Viewpoint 순찰 경로

P4 Module 3 Gas / Safety

가우시안 플룸 · Source Seeking
Return to Home (P2에 실질 의존)

※ P5는 최종이되 규격은 선행

토픽/QoS는 P3 착수 시 초안 확정
P5에서는 검증만

P5 통신 규격 + Monitoring + 통합 시나리오

P6 보너스 ②③④

P7 검증 · 문서화

- ▶ P2가 병목: 걷지 못하면 점검도 가스 추적도 시작 불가. Module 1은 Module 2·3의 전제조건이라 55h.
- ▶ P1에 ros_gz_bridge가 들어가는 이유: Harmonic은 gz-transport. 브리지를 P2로 미루면 Elevation Map이 통째로 막힌다.
- ▶ P3/P4 병렬: 가스 확산은 시뮬레이터 무관 자체 노드(가우시안 플룸)라 자원 경합이 없다.

Module 1 3단 리스크 분해 + stage1 이중화

MPC는 실패 확률이 가장 높다. 완주 보장선과 도전선을 분리한다.

단계	내용	성격	실패 시 영향
stage1 champ_tuning	기성 4족 스택 + 지형 적응 + Trot/Crawl	완주 보장선 — 필수 요건 충족	프로젝트 중단
stage2 simple_mpc_py	SRBD 간이 MPC를 Python으로 직접 구현	이해도 확보	없음
stage3 convex_mpc_cpp	Convex MPC (MIT Cheetah) C++ · 타임박스 10h	도전	없음 · 보고서에 시도·한계

Jazzy 전환으로 생긴 새 리스크: CHAMP는 Humble 기준이라 Jazzy에서 동작한다는 보장이 없다.
완주 보장선이 흔들리면 프로젝트 전체가 흔들린다. 그래서 stage1을 이중화했다. 판정은 P2 착수 전(P0 말~P1 초).

1-A 우선 경로

CHAMP 커뮤니티 Jazzy 포크를 검증한 뒤 채택.
실패하면 즉시 1-B로 전환한다.

1-B 대체 경로

gz_ros2_control + 자체 게이트 생성기
(정적 보행 궤적 + 다리 역기구학)

1-B는 손실이 아니다

발끝 궤적·역기구학 코드는 stage2 간이 MPC와 그대로 공유된다. 1-B로 가면 stage1과 stage2가 한 코드베이스로 합쳐져 총 작업량이 줄고,
“왜 직접 만들었는가”라는 심층 인터뷰 답변이 강해진다. 대신 stage1 튜닝 15h 일부가 구현으로 옮겨가므로 stage3(도전) 포기 가능성을 미리 연다.

Learning Issue 29개 — 우선순위가 학습 순서다

판단 기준 ① 다른 작업의 선행조건인가 ② 정량 목표에 직결되는가 ③ 실패 시 대체 수단이 있는가

P0 15개

없으면 프로젝트가 멈춤
Phase 0~1에 집중 학습

P1

품질·정량 목표를 좌우
해당 Phase 착수 직전에

P2

보너스·심화
Phase 6 직전에

P0 핵심 (신설 4개가 v2 대비 가장 큰 변화)

LI-01	환경	Docker arm64 Jazzy 이미지 · Foxglove/noVNC
LI-04	시스템	ros_gz_bridge / ros_gz_image ★신설
LI-12	제어	자체 게이트 생성기 (stage1-B) ★신설
LI-15	시스템	Nav2 Jazzy · costmap · Behavior Tree
LI-20·21	시스템	Gaussian Plume · Source Seeking / Infotaxis

LI-03	환경/제어	Humble→Jazzy 이식성 · 포팅 판정 절차 ★신설
LI-07	환경/비전	Harmonic thermal 센서 · heat signature ★신설
LI-08·09	제어	PointCloud 필터 · Elevation Map · 발 디딤점
LI-16·18	비전	게이지 독해 · 열화상-RGB Extrinsic 정합
LI-23	시스템	TF2 · use_sim_time · message_filters

삭제: “Gazebo 커스텀 센서 플러그인 작성법” — 네이티브 thermal 사용으로 불필요.

P1 예시: LI-13 MPC, LI-14 Fall Recovery, LI-17 저조도, LI-24 rosbridge. P2: LI-26~29 RL·다중로봇·EC2.

트랙별 배분 · 담당 산출물 · 학습 정리 템플릿

이름	담당	맞닿는 트랙	트랙	산출물
도훈	Locomotion control	제어 LI-08~14	환경	docker/ simulation/
운학	Inspection AI	비전 LI-16~18	제어	module1_locomotion/
태우	순찰 경로 · Critical Path	비전 LI-19 · /patrol_mission	비전	module2_inspection/
수현	Gazebo Virtual Plant + 관제	환경 LI-05~07 · 시스템 LI-24	시스템	module3_gas_safety/ dashboard/ 통신 규격
채현	Gas / Safety	시스템 LI-20~22	공통	REPORT.md Sim2Real

docs/learning/LI-XX.md

1. 핵심 개념 요약 (5줄 이내)
2. 우리 코드의 어디에 들어가는가
파일 경로 / 입력 토픽 / 출력 토픽
3. 선택한 방법과 근거 (대안 대비)
4. 예상 어려움 / 미해결 질문
5. 출처 (URL · 논문 · 접근일)
6. 이 LI가 답하는 동료평가 질문 번호

규칙 두 가지

- ① 개념 요약보다 “우리 코드의 어느 파일에 어떻게 들어가는가”를 반드시 한 줄 이상.
- ② 6번(대응 평가 질문)을 채운다.
이 문서가 그대로 심층 인터뷰 답변 초안.

태우의 Viewpoint/TSP(LI-19)는 운학과
/patrol_mission Action · 비용함수 입력을 P3 전에 맞춘다.

Phase 로드맵 205h + 보너스 60h · 평가 지표

Phase	시간	완료 기준	평가
P0 개발환경	15h	compose up → 빈 월드, RTF ≥ 0.8	—
P1 Virtual Plant	25h	최소 모델링 + ROS에서 전 센서 수신	DT-1
P2 Locomotion	55h	험지·계단 통과, 넘어짐 시 자동 기립	M1-1~3
P3 Inspection	29h	이동 중 판독 오차 $\leq 5\%$, 과열 검출	M2-1~3
P4 Gas/Safety	30h	임의 누출원 추적, 비상 시 복귀	M3-1,2
P5 통합+관제	30h	스크립트 1개 시나리오 + 대시보드	통합·DT-2
P6 보너스	60h	RL · 다중로봇 · 클라우드 관제	DT-3
P7 문서화	15h	제출 요건 7종	전체-1,2
예비	6h	stage1 이중화 버퍼 (P3 절감분)	—

정량 목표 (P7에서 측정)

빈 월드 RTF	≥ 0.8
통합 시나리오 RTF	≥ 0.6
험지 넘어짐 / 10회	≤ 1 회
Fall Recovery	$\geq 90\%$
게이지 판독 오차	$\leq 5\%$
열화상-RGB 정합	≤ 10 px
누출원 탐지 / 거리	$\geq 80\%$ / ≤ 1 m
Return to Home	100% (20%)
대시보드 E2E 지연	≤ 500 ms

시간 배분의 이유

P2 55h가 최대: 보행이 나머지 모듈의 전제조건. P3는 thermal 플러그인 자작 최소로 6h 절감 → 예비 6h로 P2(stage1 이중화)에 이관.
P1은 ros_gz_bridge(+3h)와 발열 오브젝트 태깅(+2h)이 늘어 25h. Phase 완료 기준을 통과하기 전에는 다음 Phase에 착수하지 않는다.
데모 영상은 각 Phase 완료 시점에 바로 녹화한다. (P2 험지 / P3 게이지+퓨전 / P4 추적 경로 / P5 통합+대시보드)

리스크 관리 + 보너스 범위 결정

#	리스크	대응
1	CHAMP 등 4족 오픈소스 Jazzy 미지원	포크 검증 → 실패 시 1-B. 궤적을 stage2와 공유. P2 전 판정
2	Apple Silicon Gazebo 성능 저하	물리 스텝 조정, headless+Foxglove. 최악 시 EC2 CPU
3	ros_gz_bridge 이미지 지연	ros_gz_image 압축, 필요 토픽만 선별 브리징
4	stage3 C++ MPC 미완성	영향 없음(stage1이 요건 충족). 타임박스 10h
5	제공 에셋 포맷 비호환	Blender 변환 우선. 불가 시 자체 모델링(과제 허용)
6	RL 정책의 Gazebo 이식 실패	도메인 랜덤화. 실패 시 학습 곡선만 보고
7	EC2 비용 초과	스팟 + 자동 중지 + \$30 예산 알람
8	환경 재전환 유혹	Phase 2 이후 Isaac Sim 전환 금지

보너스 범위

① 실시간 대시보드

필수에 흡수 (P5)
Monitoring이 이미 필수

③ RL 보행 수행 30h

EC2 스팟. Sim2Sim 이식이
DT-3(Sim2Real)의 실전 답변

② 다중 로봇 수행 15h

네임스페이스 분리만으로
구현 가능. 투자 대비 효과

④ 클라우드 관제 15h

① 대시보드를 EC2에 배포.
한계비용이 낮음

협업: Git Flow (main / develop / feature/module{n}-{task}) · Issue+Kanban · 주 1회 통합 빌드 + LI 세미나 10분.

의사결정 기록: Phase 0의 Jazzy 전환처럼 계획을 바꾼 판단은 docs/decisions/에 실측 근거와 함께 남긴다. REPORT.md 소재로 직결.

ROS 2 통신 구조 — Harmonic 브리지가 핵심

A-0 브리지 계층 (v3 신설 — Classic에는 없던 단계)

Gazebo Harmonic 센서는 gz-transport로 발행된다. ROS 2 노드는 직접 구독할 수 없다.

[gz-transport] → ros_gz_bridge (LiDAR, IMU, JointState, Clock, cmd_vel)

→ ros_gz_image (RGB, Depth, Thermal · 압축) → [ROS 2 Jazzy] sensor_msgs

설계 원칙: 필요 토픽만 선별 브리징. /clock은 gz→ROS 단방향. 전 노드 use_sim_time: true.

방식	사용 대상	이유
Topic	센서 스트림, 상태, 점검 결과	다수 구독 · 연속·비동기
Service	/set_gait_mode, /trigger_gauge_read	즉시 응답, 짧은 요청-응답
Action	/patrol_mission, /seek_gas_source /return_to_home, /fall_recovery	수 초~수 분, 진행률·취소

QoS 한 줄

센서 스트림

BestEffort · 드롭 허용

점검 결과 · 상태

Reliable + TransientLocal

/gait_mode, /robot/status

늦게 붙은 대시보드도 마지막 값

열화상 이미지

L16, 9Hz · raw×resolution=K

병목·충돌 방지 (통합-1 / 통합-2)

1. 브리지 선별 — 전 토픽 브리징은 그 자체가 병목. 이미지는 ros_gz_image 압축.
2. 대역폭 분리 — 원본 이미지는 로컬 노드만, 대시보드는 CompressedImage + 결과 메시지. rosbridge throttle.
3. 실행기 분리 — 제어 루프 200Hz와 비전 추론을 별도 프로세스. 비전은 MultiThreadedExecutor + Reentrant.
4. 시간 동기화 — TF2 map→odom→base_link→{lidar, rgb, thermal, gas}. RGB 15Hz vs 열화상 9Hz는 ApproximateTimeSynchronizer.
5. 네임스페이스 — 보너스② 대비 처음부터 /robot1/... 구조. 대시보드는 수신 시각이 아니라 header.stamp 기준 정렬.

두 축만 기억하면 된다

축 1 · 슬라이드 4

계획 → 실측 → 근거 있는 변경
Humble+Classic을 종이 위에서 고수하지 않았다.
RTF 0.5를 재고, Jazzy+Harmonic으로 바꿨다.
전체-1의 1순위 후보.

축 2 · 슬라이드 6

완주 보장선과 도전선을 분리했다.
stage1을 1-A/1-B로 이중화했고,
1-B 코드는 stage2 MPC와 공유한다.
실패해도 프로젝트가 멈추지 않는다.

동료평가 질문 ↔ 오늘 슬라이드

질문	오늘 답하는 곳
DT-1 4요소 역할	슬라이드 3 매핑 표
DT-2 · 통합-1·2 전달·동기화·병목	슬라이드 11 브리지 / QoS / 실행기 분리
DT-3 Sim2Real	보너스③ Sim2Sim (PhysX→DART) + 부록 B
전체-1 가장 어려웠던 문제	슬라이드 4 Apple Silicon + 스택 전환 실측

데모 영상 확보: P2 종료 시 험지 통과 · P3 종료 시 게이지 오버레이 + 열화상 퓨전 · P4 종료 시 추적 경로 · P5 종료 시 통합+대시보드.
발표 직전에 몰리지 않게, 각 Phase 완료 시점에 바로 녹화한다.